

SiteLens

网站安全透视系统

验证型 Web 漏洞扫描器 · 指纹识别 → 漏洞验证 → 情报关联 全链路

汇报大纲

01

选题背景与考核目标

为什么做网站安全扫描器，对应哪些考核要求

02

系统设计与实现

四层架构 · 面向对象四大特性 · 爬虫技术 · 核心模块

03

测试与验证

49 项单测 + 32 项冒烟 + 真实站点实测

04

总结与展望

成果、不足与改进方向

选题背景与考核目标

为什么做这个系统

- ▶ 现代网站由数十种第三方组件拼装：框架、CDN、统计、支付...任一组件的历史漏洞都可能成为入侵入口
- ▶ 全球漏洞情报每天新增数十条，人工翻源码、背 CVE 的评估方式效率低、易漏判
- ▶ 信息安全专业需要一套真实可用的授权评估工具，而非演示玩具
- ▶ 选题对应考核示例四「安全扫描器（综合）」并做了深度扩展

对应课程考核要求

封装 / 继承 / 多态 / 组合

检测器继承体系 + 组合根引擎

requests 爬虫技术

会话管理 · 限速 · BeautifulSoup 解析

综合运用与扩展

PostgreSQL · 异步任务 · Web 工作台 · 导出

合法合规

SSRF 防护 · 限速 · 仅限授权目标

系统定位：一次输入，完成一次安全评估

看清

被动指纹识别

7 检测器 × 2850+ 指纹
响应头/Cookie/meta/DOM/脚本



验证

漏洞主动验证

2613 条社区模板 + 42 条无害检测
发请求证实，命中才叫已验证



定论

情报三级判定

版本区间比对 + KEV 红标
评分与中文修复建议

结果全量存入本地 PostgreSQL：可检索、可对比、可导出 —— 结论逐条附原始证据，可人工复核

2850+

产品指纹

2613

验证模板

11024

漏洞情报

1695

KEV 在野利用

49+32

项测试全过

总体架构：四层设计

展示层

首页 · 扫描工作台 · 审计 · 批量 · 历史 · 情报库 · API 文档（零构建手写前端）

应用层

Flask 页面路由 · REST API（14 组） · 异步任务队列（3 并发信号量）

核心层

目标校验 · 采集解析 · 指纹检测 ×7 · 验证引擎 · 情报关联 · 网络层 · 审计 · 爆破

数据层

PostgreSQL 18: 指纹库 / 漏洞情报 / KEV / CVE / 扫描结果 / 任务队列

技术栈 Python 3.10 · Flask 3 · requests · BeautifulSoup4 · psycopg2 · PostgreSQL 18 · 零构建前端

面向对象设计：四大特性的真实落点

封装

`models.py` 全部实体类

双下划线私有属性 + 只读 property
置信度随独立证据来源自动提升（受控写入）

继承

`BaseDetector` → 7 个子类

抽象模板方法 `detect()`
子类各消费一类证据：头/Cookie/meta/DOM/脚本/表达式/bundle

多态

`engine._run_detectors()`

对检测器列表统一调用接口
新增检测器 = 新子类 + 注册一行，引擎零改动

组合

`ScannerEngine` 组合根

持有 `Fetcher/Crawler/Registry/VulnMatcher`
`Fetcher` 内组合 `RateLimiter` 限速器

爬虫技术：采集、解析与自我保护

01 requests.Session 统一采集

全局限速 0.4s · 超时重试 · UA 自报 SiteLens/1.0 · Cookie 会话注入支持认证扫描

02 BeautifulSoup 结构化解析

meta / script / DOM 选择器 / CSS 类名 / favicon mmh3 哈希 → 结构化证据对象

03 同域 BFS 深度爬取

最多 4 页合并证据，先解析 robots.txt 遵循 Disallow，子页面组件检出率显著提升

04 SSRF 防护（工具自身安全）

协议白名单 + 主机黑名单 + DNS 解析逐 IP 校验，拒绝内网/环回/保留地址与 DNS 重绑定

核心模块：被动指纹识别

2850+

产品指纹

78

检测类别

7

检测器（多态）

<1s

单页全量匹配

证据驱动 + 规则数据化：

370 条自建精编指纹（JSONB 六类规则）+ 2481 条社区表达式指纹（组内 AND、组间 OR、!= 取反，导入期预解析）

多来源独立命中自动合并，置信度每来源 +5（上限 100）；版本号从响应头 / generator / 脚本文件名正则捕获

BundleDetector 补齐「打包进产物就消失」的框架：React / Vue / Framer Motion 等 15 种库签名

核心模块：验证型检测引擎

MoE 式模板三路调度

① tag 硬匹配置顶

识别出 WordPress → WP 专项模板优先

② 语义向量路由

TF-IDF 哈希向量余弦排序（内容寻址）

③ 轮转游标兜底

无信号时轮转，2613 模板长期全覆盖

每轮仅激活 80 条模板 → 不做无差别空跑

结论可信度设计

- ▶ 只读无害：42 条自研 check 全部只读（.git 泄露 / 备份文件 / Actuator 暴露 / 目录列表...）
- ▶ 软 404 基线：先请求随机不存在路径，与基线相同的响应直接跳过
- ▶ 二次确认：命中后立即重放，两次均命中才采信，排除 CDN 抖动假阳性
- ▶ 附带轻量 DAST：反射 XSS 标记回显 / 报错型 SQLi / 开放重定向 / 目录遍历

核心模块：漏洞情报三级判定

确认受影响

版本号落在 OSV 受影响区间

例：Next.js 13.5.1 → CVE-2025-29927

可能受影响

同名组件但无版本号，仅提示

不下结论，供人工跟进

不受影响

有版本但落在区间外

直接过滤不输出，抑制误报

数据底座： 11024 条漏洞情报 · 1323 条 OSV 版本区间 · 1695 条 CISA KEV 在野利用（命中红标） · 34931 行微软公告，24h 守护线程自动更新

语义化版本比较：支持 $\geq 8.3, < 8.3.7$ / $2.x$ / $*$ 全部区间形态；检索 = trgm 粗筛 + 向量余弦重排（亚秒级）

系统测试：三层验证体系

49

项单元 / 集成测试

unittest 离线可重复

32

项 API 冒烟测试

页面 / 参数 / 边界 / 导出

14

项真实站点识别

text-well.com 全面识别

4

项靶场回归

命中 100% · 干净站零误报

测试发现并修复的真实问题：

扫描级扩展信息被子模块覆盖 → 报告缺项 · 嵌于 Next.js 流式载荷中的内联组件漏检（补 body 正则指纹） · 中文 Windows nslookup 编码不一致 → 邮件安全误报

安全工具的纪律：合规内建于代码

SSRF 防护

DNS 解析后逐 IP 校验

扫描器自身不会被当成访问内网的跳板

礼貌访问

0.4s 限速 · UA 自报 · robots 遵循

对目标的压力可控、行为可识别

POC 不落盘

导入期即丢弃 rules / payload

只做情报关联，不执行攻击载荷

授权门禁

登录爆破独立模块 + 授权声明

高危能力默认关、显式确认才可用

总结与展望

做到了

- 面向对象结构清晰：三次能力升级引擎零改动
- 「指纹 → 验证 → 情报」全链路完整，三级判定控制误报
- 工程化程度高：异步任务 / 批量 / diff / 四格式导出 / Token API
- 真实可用：数据全在本地 PG，非演示玩具

不足与改进

- 吞吐量与 Go 系引擎有量级差距 → 模板请求按 path 聚类提速
- requests 栈检不出需浏览器渲染的组件 → 明确的能力边界
- 无 OOB 回连，盲注类检不出 → 计划自托管轻量回调服务
- 验证引擎计划增加 extractor，抽取版本回灌情报判定

恳请各位老师批评指正

现场演示环节：扫描工作台 · 指纹识别 · 已验证漏洞 · 情报三级判定 · 源码审计